

ECE444 PRA3 Flask and Docker

Goal

After this session, the students should be able to:

1. Install Flask and implement an interactive HelloWorld example, and understand the basics of how Flask works, templates, web forms and connecting to Databases.
2. Download and install Docker.
3. Pull and run the updated version of the Flask Activity on their localhost.
4. Build and run Docker images of the Flask Activity on their localhost.
5. Implement a simple chatbot with memory, using Flask's session functionality.

You may use an AI coding assistant. Regardless of how you complete the work, you are responsible for testing your website and understanding all submitted changes.

Resources

"Flask Web Development: Developing Web Applications with Python" from O'Reilly, available in [UofT Library](#) and Quercus/Files/Book/Flask (Chapter 1-5)

(Referred as "textbook" below)

Tutorial of Docker:

- <https://codefresh.io/blog/hello-whale-getting-started-docker-flask/>
- <https://docs.docker.com/get-started/>
- Docker Image vs Container: Everything You Need to Know (<https://stackify.com/docker-image-vs-container-everything-you-need-to-know/>)

Preparation

First, students watch the recording of [an brief introduction to Flask](#) first, then start to read the book and play with the example.

Next, students watch the [recording of introduction to Docker](#). (Please make sure you access it with your UofT account). TAs will help students with questions during the in-person PRA sessions.

Flask Web Development

Activity 1.1 Installation

Follow Chapter 1 and install [virtual environment](#) and [Flask](#) on your local machine.

Notes:

- It writes in the textbook that windows users need to download the installer script to install virtualenv. With the recent versions of pip, virtualenv can be directly installed using command “python3 -m pip install --user virtualenv”. (See [here](#).)
- For some Windows users, simply “pip --install XXX” may not work. Try “python3 -m pip --install XXX”. (See [here](#).)
- If you are following instructions on Page 4 for Windows, please use the link to get the ez_setup.py. Link: https://bootstrap.pypa.io/ez_setup.py

E.g., Activate the virtual environment in the directory created for this PRA and import flask in this virtual environment.

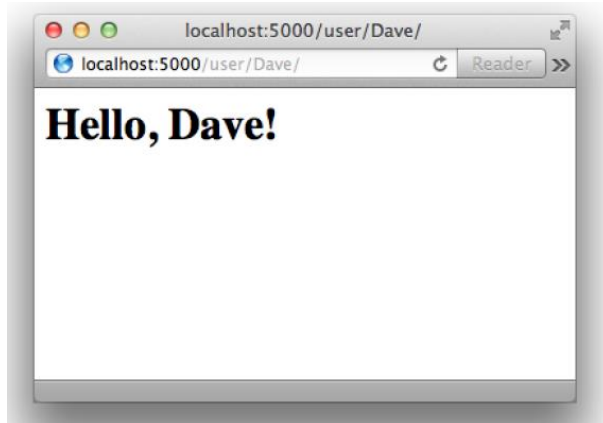
```
(venv) (base) shuruiz-mbp:flasky shurui$ date
Sat Sep 19 08:37:15 PDT 2020
(venv) (base) shuruiz-mbp:flasky shurui$ python
Python 3.7.6 (default, Jan 8 2020, 13:42:34)
[Clang 4.0.1 (tags/RELEASE_401/final)] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import flask
>>>
```

Activity 1.2 Create Your Repository and Replay Textbook Example

The learning goal is to reproduce the example from the textbook. You start with an empty repo and then commit code from the textbook example to your own repo. The textbook provides all the source code of each example: <https://github.com/miguelgrinberg/flasky>

1. Create an empty public repository, name format: E444-F2026-PRA3

2. Create a readme file with your name and mention: this repo is a clone of <https://github.com/miguelgrinberg/flasky>. (Because your repo is public on GitHub, and mentioning the source of the code is a good practice of reusing open-source code.)
3. Reproduce example 2-1 in your repo, commit the code with meaningful commit message.
4. Reproduce example 2-2 based on example 2-1 code. Commit code changes with meaningful commit message to summarize the change.



5. Push to GitHub

Activity 1.3 Replay and modify example in Chapter 3

Follow Chapter 3 examples and reproduce the screenshot below. That is, you should have at least the 3 elements from top to bottom on your webpage:

1. a navigate bar on the top
2. “Hello [your name]!” as the title
3. a timestamp in ‘LLLL’ format.

Add a screenshot for the commit on GitHub in the README file. Remember to commit the code with reasonable commit messages.

Notes:

When importing bootstrap, “from flask.ext.bootstrap import Bootstrap” is a deprecated way. Try this and see the explanation [here](#).



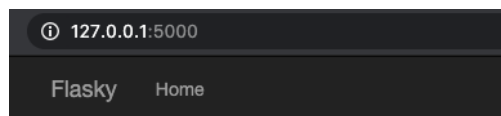
Hello Shurui!

The local date and time is Saturday, September 26, 2020 11:34 PM.

That was 3 minutes ago.

Activity 1.4 Replay and modify example in Chapter 4

1. Reproduce example 4-7 in your repo, commit the code with reasonable commit messages.
2. Add one more field for filling in email address. Please check if the email address is an UofT account ('utoronto' is a substring of the email address). If the email is an UofT address, after submitting, the browser displays the username and the email address. Otherwise, remind the user to fill in a UofT email (see screenshot below). Commit the code with reasonable commit messages. Run the project.



Hello, Stranger!

What is your name?

What is your UofT Email address?

Submit

3. Fill in your first name and your UofT email address, click submit

127.0.0.1:5000

Flasky Home

Hello, Shurui!

Your UofT email is shuruiz@ece.utoronto.ca

What is your name?

What is your UofT Email address?

Submit

4. Fill in your first name and last name in the 1st field and fill in your first name in the 2nd field, click submit. You will receive an error message

What is your name?

What is your UofT Email address?

Submit

! Please include an '@' in the email address. 'shurui' is missing an '@'.

5. Fill in your first name and last name in the 1st field and fill in your NON UofT email in the 2nd field, click submit. Make a screenshot and add it for the commit on GitHub in the README file.

Looks like you have changed your name!

Looks like you have changed your email!

Hello, Shurui Zhou!
Please use your UofT email.

What is your name?

What is your UofT Email address?

Submit

Docker

Activity 2.1 Create the PRA3_2 Branch and Pull Code

For this Activity, reuse the repo you used above.

- Go to the local directory where you cloned the GitHub repo used for PRA3-Flask. Make sure the git status is clean, which means all the changes have been added and committed.
- Create a new branch on GitHub named “PRA3_2”. Push your local PRA3_2 branch to GitHub. (“git push -u origin PRA3_2”)
- On GitHub, set the “PRA3_2” as the default branch.

Activity 2.2 Install Docker

- Follow the instructions in <https://docs.docker.com/get-docker/> to install Docker.

```
Keerthis-MacBook-Pro:~ keerthinelaturu$ docker version
Client:
 Cloud integration: 1.0.17
 Version:          20.10.8
 API version:       1.41
 Go version:        go1.16.6
 Git commit:        3967b7d
 Built:             Fri Jul 30 19:55:20 2021
 OS/Arch:           darwin/amd64
 Context:           default
 Experimental:       true

Server: Docker Engine - Community
 Engine:
  Version:          20.10.8
  API version:       1.41 (minimum version 1.12)
  Go version:        go1.16.6
  Git commit:        75249d8
  Built:             Fri Jul 30 19:52:10 2021
  OS/Arch:           linux/amd64
  Experimental:      false
 containerd:
  Version:          1.4.9
  GitCommit:        e25210fe30a0a703442421b0f60afac609f950a3
 runc:
  Version:          1.0.1
  GitCommit:        v1.0.1-0-g4144b63
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

Activity 2.3 Prepare the Application

Modify your project for PRA3 Activity 1.4:

- Change “Hello [your name]!” to “Hello [your name]! Welcome to PRA3 Docker!” as the title of your webpage.
- Commit the changes.

This step is to ensure the code base works well before you dockerize it.

Activity 2.4 Build and Run the Docker Image

1. Create and add new file “Dockerfile” into the root folder of your repository.
2. Create and add new file “requirements.txt” into the root folder of your repository.
3. Build and run docker images with the Docker commands.
4. Commit the changes.

To show the containers currently running, use **docker ps -a**, you should see the log information indicating the running application.



CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAME
8a57a1b3a836	python-docker	python3 -m flask run.	13 seconds ago	Up 12 seconds	0.0.0.0:5000->5000/tcp	teresting_carver

- The application will be running on <http://localhost:5000>.

Activity 2.5 Implement a Chatbot with Memory

Use the application from Activity 2.4 as the starting point. Add a simple chatbot that remembers information across requests in the same browser session.

1. Once a name and valid UofT email are submitted, route the Flask application to a new webpage with simple chatbot functionality. Include a message input, a Send button, and an area that displays the chatbot’s replies.
2. Given this starter endpoint below, modify the code so that the chatbot remembers something given by the user earlier in the conversation, and uses it in a later reply. Explore Flask’s session functionality and determine how to implement this behavior yourself.

```
@app.route("/chat", methods=["POST"])
def chat():
    message = request.json["message"]
```



```
if "hello" in message.lower():
    reply = "Hello!"
else:
    reply = "I don't understand."

return {"reply": reply}
```

3. The chatbot should be able to demonstrate memory across at least two requests.

For example:

```
User: My name is Alice.
Bot: Nice to meet you, Alice!

User: What is my name?
Bot: Your name is Alice.
```

4. Add a Logout button that returns the user to the Home page and clears the chatbot's remembered information without restarting the application.
5. Test that the chatbot recalls the information before logging out. Then log out, return to the chat webpage through the name and email form, and confirm that it no longer recalls the earlier conversation.

Be prepared to explain **where the remembered information is stored and how Flask knows that two requests belong to the same user.**

Submission

Submit the following on Quercus:

- Your GitHub repository URL

TA Evaluation

Students should follow the Activities (listed below) and show the completion of the tasks to the TA in their assigned PRA. It is your responsibility to ensure that the TA evaluates your PRA submissions before the conclusion of the PRA session.

The TA will:

- Verify the commits
- Select one or more commits from your history and ask you to explain the changes made in those commits
- Ask questions about Flask or Docker concepts practiced in the activities in this PRA.

This PRA is worth 4 points:

- Completion and correctness of Activities: 2 points
- Explanation of selected commits and demonstrated understanding: 2 points

Notes:

1. You can use the latest [official documentation of flask 3.1.3](#) as another source of reference since our textbook was released in 2014. Also pay attention that the version of Python in the textbook is 2.7/3.3. You are encouraged to use the latest version of Python and Flask on your PC but just remember to take version difference into consideration when you encounter some confusing issues.
2. Please pay attention to **notes** in every activity.
3. For Windows users, Command Prompt (CMD) is recommended (rather than PowerShell).